LLM 모델 선정, 지식 베이스 및 RAG 구축 이론 정리

1. LLM이란?

정의

LLM(Large Language Model, 대규모 언어 모델)은 방대한 텍스트로 학습해 다음에 올 단어(토큰)를 예측하는 방식으로 문장을 이해하고 생성하는 인공지능 모델입니다. 질의응답, 요약, 번역, 분류, 코드 작성 등 다양한 언어 작업을 하나의 모델로 처리합니다.

특징

트랜스포머(Transformer) 구조를 기반으로 하며, 파라미터 수가 수십억~수조 개에 이릅니다.

프롬프트(지시문)만 바꿔도 여러 작업을 수행할 수 있습니다.

한 번에 처리할 수 있는 입력 길이(컨텍스트 윈도우)가 정해져 있습니다.

같은 질문에도 샘플링 설정(temperature 등)에 따라 답이 달라질 수 있습니다.

한계

환각(Hallucination): 근거 없이 그럴듯한 내용을 만들어 낼 수 있습니다.

최신성 부족: 학습 시점 이후의 정보는 모릅니다.

내부 지식 부재: 학습하지 않은 사내 문서나 개인 데이터는 답할 수 없습니다.

출처 불명확: 답변의 근거를 스스로 제시하기 어렵습니다.

RAG와 지식 베이스는 이 한계 중 최신성, 내부 지식, 출처 문제를 줄이기 위한 대표적인 방법입니다.

2. LLM 모델 선정

정의

모델 선정은 서비스의 목적, 품질 요구, 비용, 보안 조건을 기준으로 가장 적합한 LLM을 고르는 과정입니다. 가장 큰 모델이 항상 최선은 아니며, 요구 품질을 만족하는 가장 작은 모델을 찾는 것이 기본 원칙입니다.

선정 기준

기준	확일한 내용
성능	목표 작업(질의응답, 요약, 추론, 코드)에서의 정확도
한국어 능력	한국어 이해·생성 품질, 토큰 효율
컨텍스트 길이	검색 결과와 대화 이력을 함께 넣을 수 있는 입력 길이

비용	입입력·출력 토큰당 요금 또는 자체 GPU 운영비력·출력 토큰당첫 토큰까지의 지연 시간, 초당 생성 토큰 수 요금 또는 자데이터 외부 전송 허용 여부, 서비스 위치, 개인정보 처리체 GPU함수 호출(Tool use), 구조화 출력(JSON), 멀티모달 지원 운영비
응상업적 이용 가능 여부, 사용 조건답 속도	첫 토큰까지의 지연 시간, 초당 생성 토큰 수
보안·규정	데이터 외부 전송 허용 여부, 서비스 위치, 개인정보 처리
기능	함수 호출(Tool use), 구조화 출력(JSON), 멀티모달 지원
라이선스	상업적 이용 가능 여부, 사용 조건

상용 API 모델과 오픈 모델

구분	상용 API 모델	오픈 가중치 모델
운영 방식	제공사 API 호출	직접 서버·클라우드에 배포
초기 구축	빠르고 쉬움	GPU 확보와 운영 인력 필요
성능	일반적으로 최상위 성능	크기와 종류에 따라 다양함
데이터 통제	외부 전송을 계약·설정으로 관리	내부망 운영 가능
커스터마이징	프롬프트 중심, 일부 파인튜닝 지원	파인튜닝과 양자화 등 자유도 높음
비용 구조	사용량 기반	고정 인프라 비용

선정 절차

사용 사례와 성공 기준(정확도, 지연 시간, 월 예산)을 정합니다.

보안·규정 조건으로 후보를 거릅니다.

실제 질문 50~200개 정도로 평가 세트를 만듭니다.

후보 모델 2~3개를 같은 프롬프트로 비교합니다.

품질, 비용, 속도를 종합해 선택하고, 운영 중 재평가합니다.

평가 방법

공개 벤치마크: 후보를 좁히는 참고 자료로만 씁니다. 실제 업무 품질과 다를 수 있습니다.

자체 평가 세트: 서비스에서 실제로 나올 질문과 기준 답변으로 만듭니다.

LLM-as-a-judge: 다른 LLM이 답변을 채점하게 하는 방식이며, 사람의 검토로 보완합니다.

운영 지표: 응답 시간, 비용, 사용자 피드백을 지속적으로 측정합니다.

3. 지식 베이스 구축

정의

지식 베이스(Knowledge Base)는 LLM이 답변 근거로 참조할 문서와 데이터를 검색하기 좋은 형태로 정리해 저장한 저장소입니다. 정형·반정형·비정형 데이터가 모두 대상이 될 수 있으며, 문서 품질이 곧 답변 품질이 됩니다.

구축 단계

데이터 수집: PDF, 워드, 웹 페이지, 위키, 티켓, DB 테이블 등 대상 자료를 모읍니다.

정제·전처리: 머리글·바닥글, 중복, 깨진 문자를 제거하고 표와 이미지는 텍스트로 변환합니다(OCR 포함).

메타데이터 부여: 출처, 작성일, 부서, 문서 유형, 접근 권한 등을 붙여 필터링과 출처 표시에 씁니다.

청킹(Chunking): 문서를 검색 단위로 나눕니다.

임베딩(Embedding): 각 청크를 의미를 담은 숫자 벡터로 바꿉니다.

색인·저장: 벡터 DB 또는 검색 엔진에 벡터와 원문, 메타데이터를 함께 저장합니다.

갱신 관리: 문서가 바뀌면 해당 청크만 다시 색인하고, 삭제된 문서는 제거합니다.

4. RAG 구축

정의

RAG(Retrieval-Augmented Generation, 검색 증강 생성)는 사용자 질문과 관련된 문서를 지식 베이스에서 먼저 검색하고, 그 내용을 프롬프트에 포함해 LLM이 근거에 기반해 답하게 하는 구조입니다.

동작 흐름

질문 입력: 사용자가 질문합니다.

질문 처리: 필요하면 질문을 다시 쓰거나(쿼리 재작성) 대화 이력을 반영합니다.

검색(Retrieval): 질문을 임베딩해 지식 베이스에서 유사한 청크 상위 k개를 찾습니다.

재순위화(Reranking): 검색 결과를 더 정밀한 모델로 다시 정렬합니다.

프롬프트 구성: 지시문, 검색된 근거, 질문을 하나의 프롬프트로 만듭니다.

생성(Generation): LLM이 근거를 바탕으로 답변을 작성합니다.

출처 표시·검증: 답변에 사용한 문서를 함께 제시하고, 근거가 없으면 모른다고 답하게 합니다.

프롬프트 구성 원칙

제공된 근거 안에서만 답하도록 지시하고, 근거가 부족하면 그렇게 말하게 합니다.

근거마다 번호와 출처를 붙여 인용하게 합니다.

컨텍스트 윈도우를 넘지 않도록 검색 결과 수와 길이를 제한합니다.

고도화 기법

쿼리 재작성·확장: 모호한 질문을 검색에 적합한 문장으로 바꿉니다.

메타데이터 필터링: 부서, 기간, 권한으로 검색 범위를 좁힙니다.

상위 문맥 확장: 작은 청크로 찾고 주변 문단까지 함께 전달합니다.

다단계·에이전트형 RAG: 필요할 때 여러 번 검색하거나 도구를 호출합니다.

그래프 RAG: 개체 간 관계를 그래프로 저장해 여러 문서에 걸친 질문에 대응합니다.